

ACE-ASP: A Controlled Fragment of Attempto Controlled English for Answer Set Programming

Language Design, Semantic Reinterpretation and Syntax-Directed Translation via
CNLWizard

1 From ACE to Answer Set Programming: Analysis of the Semantic Mismatch

Attempto Controlled English (ACE) is a general-purpose controlled natural language whose reference semantics is full first-order logic. In the canonical Attempto toolchain, a text is processed by APE (Attempto Parsing Engine), a comprehensive Prolog-based parser that resolves anaphora and scope ambiguities through native linguistic heuristics and emits a Discourse Representation Structure (DRS), an intermediate logical layer of discourse referents and atomic conditions. Any translation of ACE into a target formalism is traditionally defined as a transformation of this DRS.

Answer Set Programming occupies a different point in the design space of logical languages. On the one hand it is strictly poorer than first-order logic: programs are function-free, and no existential quantification is permitted in rule heads. On the other hand it is strictly richer: it distinguishes two negations (classical negation and negation as failure under the stable model semantics), and it offers native constructs with no first-order counterpart, such as choice rules, cardinality aggregates and weak constraints. A useful ACE fragment for ASP therefore cannot be obtained by pruning alone. It must be defined by two complementary moves:

- **Syntactic subtraction.** Starting from the inventory of the ACE 6.7 Construction Rules, every construction is classified as *kept*, *restricted* or *dropped*. Every ACE-ASP sentence is, by construction, a syntactically valid ACE sentence, and the subset claim is verifiable rule by rule.
- **ASP-native reinterpretation.** A small number of retained constructions receive a semantics that deliberately departs from the ACE interpretation rules, chosen because it maps onto idiomatic ASP. Modal verb phrases become choice rules, hard constraints and weak constraints; generalised quantifiers become cardinality aggregates. ACE-ASP is thus a syntactic subset of ACE that is *not semantically conservative*; the deviation is declared and is, in fact, the core design contribution. The guiding philosophy is the *guess & check* methodology of ASP: linguistic expressions of possibility generate (guess), while expressions of necessity and numeric conditions restrict (check).

A second, purely architectural difference concerns the translation machinery itself. Instead of a top-down traversal of a semantically resolved DRS, integration within the CNLWizard framework imposes a direct, syntax-directed translation: the Lark parser produces an Abstract Syntax Tree that is evaluated strictly bottom-up. Leaf constituents are compiled before the sentence context that determines their eventual role (fact, rule, constraint, choice or query) is available, so the translator must capture partial semantic states and propagate them upwards until the statement can be materialised at the root. Section 5 shows how this inversion is handled through a small hierarchy of *unsaturated* semantic objects.

2 Defining the Fragment: Subtractive Grammar Design

2.1 Dropped constructions

Table 1 summarises the ACE 6.7 constructions that are absent from ACE-ASP, together with the motivation for their removal. The removals fall into three families: constructions whose semantics has no function-free counterpart (plural collective entities, mass nouns, measurement nouns, reals, strings, sets and lists); constructions whose resolution would require the linguistic machinery of APE (the entire

pronoun system and definite anaphora); and constructions that are mere syntactic sugar or would multiply predicate arity (passives, dative shift, adverbs and prepositional-phrase modifiers).

ACE 6.7 construction	Motivation for removal
Plural NPs (collective and distributive), <i>there are</i>	would denote set-valued entities, which are not function-free
Mass nouns, measurement nouns	no natural ASP semantics for quantities of substance
Real numbers, exponentiation	clingo arithmetic is integer-based
Strings with &, sets { }, lists []	no native support in the target language
All pronouns; anaphoric <i>the N</i>	co-reference is expressed exclusively through ACE variables and Skolem constants (Sect. 4.6), removing DRT-style anaphora resolution
<i>not every</i> , <i>nobody/nothing but</i> , global quantification	the underlying quantifier dualities are not expressible as a single rule
Saxon genitive, possessive pronouns	ambiguous relational readings; only the functional <i>of</i> -genitive is retained
Comparatives, superlatives, <i>as..as / more..than</i>	would require #min/#max or optimisation statements
Adverbs and PP modifiers of verb phrases	each modifier would change the arity of the verb predicate
Passives, double-object ditransitives	syntactic sugar over the retained active forms
Sentence subordination (<i>believes that</i>)	would require reification of literals
Imperative sentences	no ASP counterpart
Queries <i>how/where/when/whose</i> ; multiple queries	interrogate dropped modifiers, or require variable-arity answer predicates
Redundant modal forms (<i>cannot, has to, it is necessary that, ...</i>)	one surface form is retained per ASP meaning

Table 1: Dropped ACE constructions and their motivations.

2.2 Restricted constructions

The retained grammar is subject to a small set of restrictions, each of which converts a hard semantic problem into a syntactic or lexical one:

- **Variables replace anaphora.** A variable may be used as a noun phrase only if it was previously introduced in apposition to a common noun (*a person X*) or bound to a Skolem constant by a preceding sentence. ACE's *bare variables* (“X weighs 300 kg”) are dropped: in ASP every variable needs a domain literal, so requiring an introducing noun makes rule safety an almost-syntactic property (Sect. 4.8).
- **Reserved function words.** ACE prescribes that function words are predefined and cannot be redefined by users. ACE-ASP enforces this at the lexical level: the open classes (nouns, verbs, adjectives) are token patterns that explicitly exclude the closed function-word vocabulary (Sect. 3.2).
- **Proper name shape.** Proper names are capitalised multiwords of at least two characters whose second character is a lowercase letter (*John, Mr-Miller, Node1* via *Node+digit* patterns), so that they never collide with the variable pattern `[A-Z][0-9]*`.
- **Integers only**, and boolean formulas (`=`, `\=`, `>`, `>=`, `<`, `=<`) are admitted exclusively as body literals, never as top-level sentences: a builtin comparison cannot be asserted as a fact.

- **Copula complements** are limited to noun phrases, positive adjectives (including hyphenated transitive adjectives such as *fond-of*, *assigned-to*) and ground arithmetic expressions.

3 Determinism and Lexical Engineering

ACE-ASP has no lexicon: content words are open classes recognised by token patterns. Combined with a general-purpose parser, this creates a class of tokenisation and ambiguity problems that a DRS-based pipeline never encounters, because APE resolves them semantically. This section documents the engineering decisions that make the grammar deterministic in practice.

3.1 Connector precedence

ACE prescribes that *and* binds more strongly than *or*, and provides *,and* and *,or* to override the standard binding order. Since ASP realises disjunction in bodies by generating multiple rules and conjunction by aggregating literals within one body, the AST must keep the two connectives at distinct hierarchical levels. The precedence ladder is encoded as a four-level cascade of non-terminals, each equipped with CNLWizard's `concat` mechanism:

```
sentence_body    := body_and_group  ( ", or"  body_and_group )*
body_and_group   := body_or_group   ( ", and" body_or_group  )*
body_or_group    := body_conjunction ( "or"   body_conjunction )*
body_conjunction := body_literal    ( "and"   body_literal  )*
```

Listing 1: The connector-precedence cascade (conceptual view of the generated Lark rules).

During the bottom-up traversal each level folds its children into a Disjunctive Normal Form — a list of alternative conjunctions — so that at the root every disjunct materialises as a separate rule with an identical head, while every conjunct joins the same rule body.

3.2 Word boundaries and reserved words

Two lexical phenomena had to be neutralised. First, plain string terminals carry no word-boundary information, so the determiner *A* could consume the prefix of *An employee*, leaving a spurious one-letter noun *n*; analogously *and* could be tokenised as *a+nd*. All closed lexical classes (determiners, modals, generalised quantifiers, relative pronouns) are therefore regex terminals with an explicit boundary lookahead. Second, without a lexicon the function words themselves match the open-class pattern: *is* would be a legal transitive verb, yielding a structurally valid but nonsensical parse of “Every man is a human”. The open-class terminals exclude the reserved vocabulary through a negative lookahead:

```
WORD: /(?!(?:(a|an|one|the|is|be|does|not|provably|can|may|must|should
|no|every|each|that|who|which|to|of|if|then|there|it|false|possible
|and|or|at|least|most|more|less|than|exactly)(?![a-z0-9_-]))
[a-z][a-z0-9_-]*(-[a-z0-9_-]+)*/
```

Listing 2: The open-class terminal reserving ACE function words (line breaks added for readability).

With these measures the entire regression suite parses with a single residual ambiguity, the associative one on chains of three or more coordinated constituents ((*A or B*) or *C* versus *A or (B or C)*), which is semantically immaterial because the folding functions flatten the chain.

3.3 The negation marker

An optional anonymous token such as *("not")?* is filtered out of the Lark parse tree: the ASTs of “Does John wait?” and “Does John not wait?” would be indistinguishable, and the polarity information would be lost silently. Negation is therefore reified as a dedicated `negation_marker` production, which survives in the tree and reaches the translation functions as an explicit boolean.

3.4 Numeric literals

Lark's stock **NUMBER** terminal matches floating-point forms, so in “The age of John is 32.” it consumes “32.” including the sentence terminator. All numeric positions use the integer terminal **INT**, consistently with the integers-only restriction of Section 2.2.

4 Semantic Interpretation and ASP Translation Strategy

4.1 The dispatch principle

ACE-ASP deliberately preserves ACE's uniform sentence structure: a declarative sentence is always *noun phrase* + *verb phrase*, and no grammar production distinguishes facts from rules or constraints. The kind of ASP statement is decided compositionally by two features that the bottom-up traversal carries upwards: the **determiner of the subject** and the **modality of the verb phrase**. Table 2 gives the complete dispatch.

ACE-ASP sentence	ASP statement
John waits.	<code>waits(john).</code>
There is a person X. / A man waits.	<code>Skolemised facts: person(sk1).</code>
Every man is a human.	<code>human(X) :- man(X).</code>
If a man X owns a dog Y then X likes Y.	<code>likes(X,Y) :- man(X), owns(X,Y), dog(Y).</code>
Every node is red or is green.	<code>red(X) green(X) :- node(X).</code>
No dog is a cat.	<code>:- dog(X), cat(X).</code>
It is false that A and that B. / It is not possible that ...	<code>:- a, b.</code>
Every employee is assigned-to exactly 1 team.	<code>:- employee(E), #count{T : assigned_to(E,T), team(T)} != 1.</code>
Every employee can be assigned-to a team.	<code>{ assigned_to(E,T) : team(T) } :- employee(E).</code>
... can not / may not / must not ...	<code>:- domain, p(...).</code>
Every employee must be assigned-to a team.	<code>:- employee(E), #count{T : assigned_to(E,T), team(T)} = 0.</code>
Every server should not be overloaded.	<code>:- server(S), overloaded(S). [1@1,S]</code>
Which man waits?	<code>ans(X) :- man(X), waits(X). #show ans/1.</code>
How many man waits?	<code>ans(N) :- N = #count{X : man(X), waits(X)}.</code>
Does John wait?	<code>ans :- waits(john). #show ans/0.</code>

Table 2: The sentence-level dispatch from (subject determiner, VP modality) to ASP statements.

4.2 The two negations

ACE natively distinguishes logical negation from negation as failure, and the two map one-to-one onto the two negations of ASP — the cleanest point of contact between the source and target languages. *does not* / *is not* translates to classical (strong) negation $\neg p(X)$, admissible both in bodies and heads; *does not provably* / *is not provably* translates to negation as failure $\text{not } p(X)$, admissible only in bodies. Asserting a NAF literal as a top-level fact is rejected with a message suggesting a conditional reformulation. The canonical example:

If a person P is not provably guilty then P is innocent.
 `innocent(P) :- person(P), not guilty(P).`

4.3 Conditionals, DNF bodies and disjunctive heads

The if-part of a conditional is compiled to DNF as described in Section 3.1; each disjunct yields one rule. In the then-part, *or* becomes the disjunction `|` of a disjunctive head, whereas *and* is split into several rules sharing the body; the mixed form (A and B) or C is rejected as inexpressible in a single rule. Heads obey

the fundamental restriction of Section 1: no existentials. A then-part may only mention proper names and variables bound in the if-part; “then X owns a dog” is refused with a suggestion to use the modal *can* (i.e. a choice rule) instead.

4.4 Generalised quantifiers: the dual reading

The quantifiers *at least* / *at most* / *more than* / *less than* / *exactly n* compile to `#count` aggregates under two complementary readings, selected by the syntactic position:

Position	Reading	Mapping (word → comparator)
Top-level requirement (e.g. subject <i>every</i>)	violation is forbidden	at least→<, at most→>, more than→<=, less than→>=, exactly→!=
Inside an if-body or a query	positive condition	at least→>=, at most→<=, more than→>, less than→<, exactly→=

Table 3: The dual reading of generalised quantifiers.

For instance, “Every employee is assigned-to exactly 1 team” forbids the complement of the required condition (comparator !=), whereas “If a team has at least 2 employees then ...” contributes a positive >= condition to the rule body. A subtle consequence for the architecture: when the quantified phrase is an *object*, its predicate literal must be placed *inside* the aggregate rather than in the rule body, which is why quantified objects must travel upwards to the sentence node (Sect. 5.1).

4.5 Modality as guess & check

Modal verb phrases receive the announced ASP-native reinterpretation. *can/may* generate a choice rule whose element is the predication, whose condition is the domain of the existential object, and whose body is the subject domain; a quantified object contributes cardinality bounds (*can be assigned-to at most 2 teams* ■ { ... } 2 :- ...). *must* forbids the violation of the obligation, and *should* is its soft variant, a weak constraint over the same violation body — the hard/soft symmetry keeps the language predictable. The violation body itself hides a safety subtlety: with an existential object, the naive `not assigned_to(E,T)` would leave T unsafe, so the obligation is expressed through a zero-count aggregate:

```
Every employee must be assigned-to a team.
■ :- employee(E), #count{ T : assigned_to(E,T), team(T) } = 0.

Every server should not be overloaded.
■ :- server(S), overloaded(S). [1@1,S]
```

4.6 Existentials, Skolemisation and cross-sentence variables

Top-level existential sentences (*There is a N.* and indefinite subjects of assertions) are the only place where the existential quantifier survives, and it does so through Skolemisation: every free variable of the fact set is replaced by a fresh constant `sk1`, `sk2`, ... If the noun phrase carries a variable in apposition, the binding is recorded in the translation context and the variable remains referable in subsequent sentences:

```
There is a person X.   X is tall.
■ person(sk1).   tall(sk1).
```

ACE’s cross-sentence anaphora is thereby reduced to a dictionary lookup. Equalities produced by the copula are unified *before* Skolemisation, so that “The age of John is 32.” yields the fact `age(john, 32)` rather than a spurious Skolem constant equated to a number.

4.7 The functional genitive

The retained genitive *the N of X* is read functionally as a binary predicate whose second argument is a fresh value variable. In a conditional it interacts with boolean formulas through equality binding:

```
If the age of a person P is Y and Y >= 18 then P is an adult.  
■   adult(P) :- person(P), age(P,V1), V1 = Y, Y >= 18.
```

4.8 Safety

Every assembled statement is validated against the safety rule adapted from clingo: the variables of the head, of NAF literals, of builtins and of aggregate guards must be a subset of the variables bound by positive body literals. Three refinements matter in practice: (i) a strongly negated literal $\neg p(X)$ is positive in the ASP sense and binds X; (ii) the result variable of $N = \#count\{ \dots \}$ binds N, while the local variables of the aggregate stay internal; (iii) an equality builtin with one ground side binds the other side, computed as a fixed point — which is what makes the genitive example of Section 4.7 safe, binding Y through V1. Because every variable must be introduced by a common noun (Sect. 2.2), safety violations are rare and, when they occur, are reported in natural language: “*variable X is not safe: every variable must be introduced by a common noun (e.g. 'a person X') in a positive literal of the premise*”.

4.9 Queries

A text admits at most one interrogative sentence, in final position (consistent with ACE, where interrogatives are never anaphoric antecedents). All three query forms compile to rules over a reserved predicate `ans/N` plus the corresponding `#show` directive: yes/no queries produce `ans/0` (the brave/cautious alternative is a solver option, not a language feature), *which*-queries produce `ans/1` over the domain noun, and *how many*-queries bind the answer to a count aggregate. Bare *who/what* are excluded because they would generate unsafe answer rules; *which N* carries its own domain literal.

5 CNLWizard Implementation

The implementation follows CNLWizard's two-step workflow. In the generation step, a YAML specification whose non-terminals mirror the names of the ACE construction rules (`simple_sentence`, `there_is_sentence`, `relative_clause`, `generalised_quantifier`, `boolean_formula`, ...) produces the Lark grammar and one Python function per AST node. In the compilation step, the AST is evaluated bottom-up and the per-node functions assemble the translation. The functions themselves are deliberately thin glue: the semantic intelligence lives in a separate, independently testable package, `ace_asp_lib`, organised in six modules (`terms`, `ir`, `results`, `context`, `safety`, `build`).

5.1 Unsaturated semantic objects

The central problem of the bottom-up inversion is that when a verb phrase is evaluated, its subject does not exist yet. The solution adapts a Montagovian idea: *a verb phrase is not a literal, it is a function from terms to literals*, reified for debuggability. A noun phrase evaluates to an `NPResult` — a term together with the domain literals that restrict it and a quantifier descriptor. A verb phrase evaluates to a `VPResult`, a bundle of `Predication` objects whose literals contain the placeholder `SUBJ` in subject position; the object noun phrases are stored unevaluated inside each predication, because the sentence node must still inspect their quantifiers. Saturation happens in `apply(subject)`, which substitutes the placeholder, inlines the restrictors of non-quantified objects, and returns quantified objects separately as triples (`literal`, `gq_np`, `extra`) so that the literal can be placed inside the aggregate (Sect. 4.4). Relative clauses reuse the same abstraction: a relative is a `VPResult` applied to the term of the noun phrase it modifies, its output feeding the restrictor list. This uniformity (NP = term + constraints, VP = function, relative = VP) is what keeps the code base small.

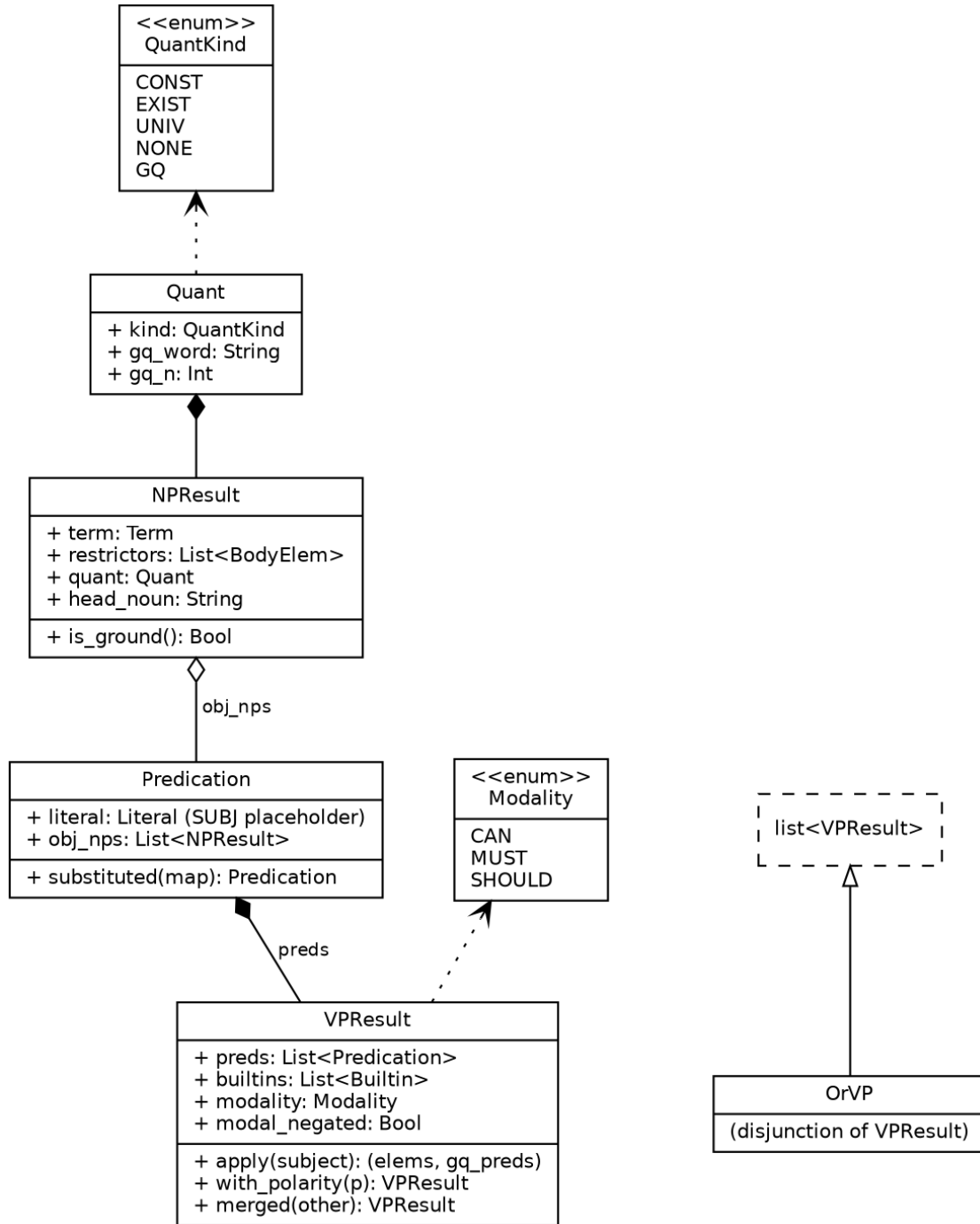


Figure 1: UML class diagram of the unsaturated semantic layer (module results). Predication aggregates the object NPResults so that quantified objects can bubble up to the sentence node.

5.2 The intermediate representation

Statements are assembled as an explicit IR, separated from rendering: tests compare structures rather than strings, and the experimental validation can feed the rendered text to clingo. `Literal` carries a four-valued polarity (positive, strong negation, NAF, NAF over strong negation); `RuleIR` encodes facts (empty body), integrity constraints (empty head) and disjunctive rules (multiple head literals) uniformly; `ChoiceIR`, `WeakIR` and `QueryIR` cover the remaining statement kinds; `CountAggregate` operates either as a guarded cardinality test or as an assignment to a result variable. Every class exposes `render()` emitting clingo syntax.

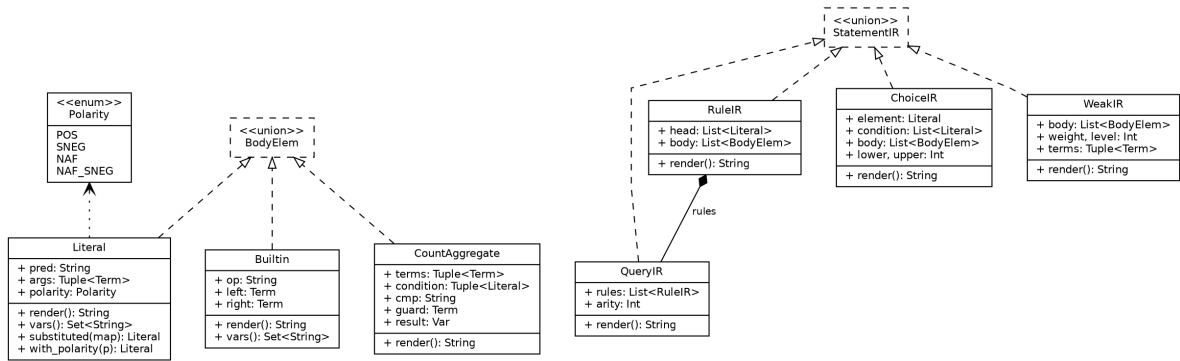


Figure 2: UML class diagram of the intermediate representation (module ir).

5.3 The translation context

Because the per-node functions are free functions invoked by the transformer, the state that must persist across sentences lives in an explicit context object with a reset hook for testing. `ConceptRegistry` plays the role that a terms dictionary plays in lexicalised CNLs, adapted to an open vocabulary: it assigns each surface word a predicate with a fixed arity, detects arity clashes across sentences (reporting them in natural language rather than letting clingo fail later), unifies verb forms, and rejects reserved words. `VarManager` issues deterministic fresh variables per sentence and tracks user variables; the Skolem table implements the cross-sentence binding of Section 4.6.

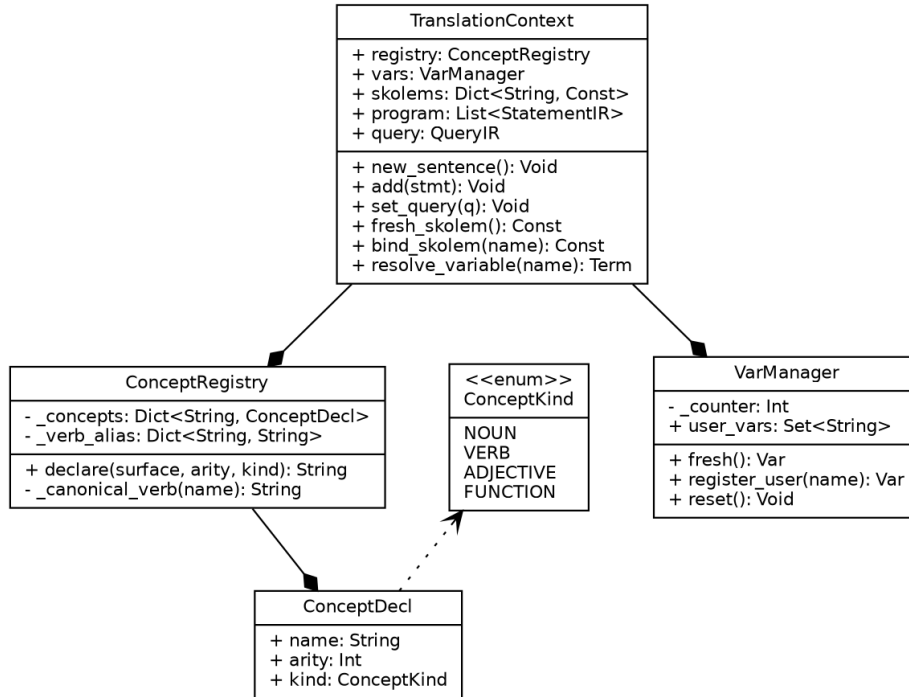


Figure 3: UML class diagram of the translation context (module context).

5.4 The statement assembler

The module `build` hosts the sentence-level semantics. `clause_to_dnf` converts an (NPResult, VPResult) pair in body position to DNF, distributing VP-disjunctions and compiling quantified constituents to positive aggregates; `head_clause_to_disjuncts` does the dual job for then-parts, enforcing the head restrictions; `add_simple_sentence` is the dispatcher realising Table 2, including Skolemisation with prior equality unification and the modal encodings of Section 4.5. Every constructed statement passes through `safety.check` before being appended to the context.

5.5 AST traversal and data flow

The overall compilation follows a five-step flow, visualised in Figure 4:

- **Lexical resolution (leaves).** Tokens become terms and surface strings; content words are registered as predicates with their arity.
- **Constituent construction.** Noun-phrase nodes build `NPResult` objects; verb-phrase nodes build `VPResult` objects around the `SUBJ` placeholder, storing object `NPResults` unevaluated.
- **Coordination folding.** The concat cascades fold conjunctions and disjunctions into DNF bodies, head disjunct lists and `OrVP` bundles.
- **Statement materialisation.** Sentence nodes finally see both the subject and the saturated predications, dispatch on (determiner, modality), assemble the IR, validate safety and append to the context.
- **Program emission.** The root node renders the accumulated statements (plus the optional query) into the final clingo program and resets the context.

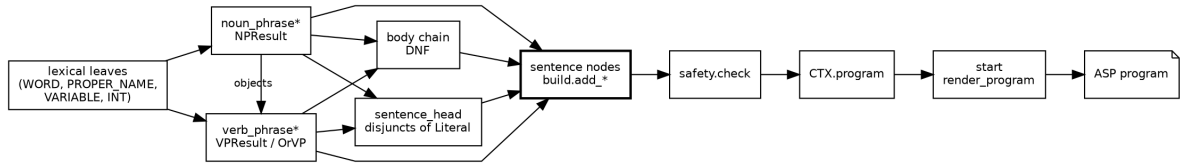


Figure 4: Bottom-up data flow from lexical leaves to the emitted ASP program.

A worked trace makes the flow concrete. For “If a person *P* is not provably guilty then *P* is innocent.”: the premise NP yields `NPResult(P, [person(P)], EXIST)`; the copula complement yields `guilty(_SUBJ)`, which *is not provably* re-polarises to NAF; `clause_to_dnf` saturates the subject producing the single conjunction `[person(P), not guilty(P)]`; the then-part resolves *P* as an already-known variable and produces the head disjunct `[innocent(P)]`; the conditional node builds the rule, the safety check confirms that *P* is bound by the positive literal `person(P)`, and the rule joins the program.

5.6 Toolchain integration notes

Two integration details deserve mention. First, CNLWizard's pre-processing stage (signature definitions and *where*-substitution) is disabled through the tool's configuration mechanism, set directly from the translation module: ACE-ASP defines its entities inside the language itself, its assertions may syntactically resemble signature definitions, and its interrogative sentences terminate with a question mark that must reach the parser untouched. Second, the grammar generator does not preserve the quantifier `?` inside `/regex/` syntax entries (it interprets it as the optionality marker), so the boundary and reserved-word lookaheads of Section 3.2 are restored by a small project-level generation script that invokes CNLWizard and repairs the eight lexical terminals in the emitted grammar. Both measures leave the CNLWizard code base untouched.

6 Validation

The system is validated at three levels, all automated. At the **grammar level**, a regression suite parses a corpus of sentences covering every construction, and an explicit-ambiguity run certifies that the only ambiguity left is the benign associative one. At the **library level**, unit tests exercise the IR, the safety refinements (strong negation binds, NAF does not, aggregate result variables bind, the equality fixed point) and the context (arity clashes, verb unification, Skolem binding, single-query enforcement), solving the rendered programs with the clingo API and checking model counts and optimal costs. At the **end-to-end level**, complete ACE-ASP texts are compiled through `CnlWizardCompiler` and solved; the

strongest test translates a graph 3-colouring specification and verifies that its answer sets are *identical* (projected on the colour predicates) to those of a hand-written encoding, six models for the triangle instance. Negative tests assert that ill-formed inputs — unsafe variables, arity clashes, existentials in heads — surface as ACE-ASP exceptions with actionable natural-language messages rather than as clingo errors.

```
Node1 is a node. Node2 is a node. Node3 is a node.  
Node1 is connected-to Node2. Node2 is connected-to Node3.  
Node1 is connected-to Node3.  
Every node is red or is green or is blue.  
It is false that a node X is connected-to a node Y and X is red and Y is red.  
It is false that a node X is connected-to a node Y and X is green and Y is green.  
It is false that a node X is connected-to a node Y and X is blue and Y is blue.
```

Listing 3: Graph 3-colouring in ACE-ASP; its answer sets coincide with those of the reference hand-written encoding.

7 Conclusion

ACE-ASP shows that a useful ASP front-end can be carved out of a general-purpose controlled language by combining disciplined syntactic subtraction with a declared, ASP-native reinterpretation of a handful of constructs. The design turns the hardest problems of the source language into tractable ones: anaphora becomes variable binding, quantifier scope becomes a positional dual reading, and rule safety becomes an almost-syntactic property backed by a validating checker with natural-language diagnostics. On the implementation side, the bottom-up inversion imposed by syntax-directed translation is absorbed by a compact layer of unsaturated semantic objects — noun phrases as constrained terms, verb phrases as reified functions over a subject placeholder — while an explicit intermediate representation decouples assembly from rendering and makes the pipeline testable down to answer-set equivalence with hand-written encodings. The resulting system covers facts, rules with both ASP negations, disjunctive heads, integrity constraints, cardinality aggregates, choice rules, weak constraints and three query forms, and provides a principled base for future extensions such as optimisation statements for superlatives and instance-generation sugar.